

Cloud-Native Observability Plane for free5GC

A working 5G core network with 20 devices, two network slices, and live monitoring

Repository: github.com/RithvikReddy0-0/5g-observability · Report generated 8 August 2026

What this project is, in one paragraph.

We built a real 5G mobile core network that runs on a laptop. It has all the pieces a real operator would have — the parts that authenticate your phone, decide which network "slice" you belong to, and manage your data session. We connected 20 simulated phones to it, split across two separate network slices, and then built a monitoring system on top so we can see what the network is doing. Everything is automated and reproducible: one person can rebuild the whole thing from scratch by running a few scripts.

1. Results at a glance

8 / 8 Network functions running and registered	20 / 20 Devices (UEs) successfully connected	2 Network slices, working independently
10 / 10 Monitoring targets healthy	12 / 12 Testable requirements passed	0 Failures

What we checked	Result	How we proved it
Software versions locked and reproducible	PASS	Deleted everything, re-downloaded — got byte-identical results
All core network functions running	8 / 8	Read the network's own registry database
Subscribers (SIM cards) provisioned	20	Database query
Devices connected to the network	20 / 20	Asked each device directly for its status
Both network slices selected correctly	PASS	60 requests for each slice in the logs
Monitoring collecting data	10 / 10	Prometheus target page
Data traffic through the network	BLOCKED	Needs different hardware — explained in section 5

2. What we built

The 5G core network

Eight "network functions" — think of them as specialised servers that each do one job. The AMF handles device connections, the AUSF and UDM check that a SIM card is genuine, the SMF sets up data sessions, and the NSSF decides which network slice a device belongs to. They find each other through a registry called the NRF, and all their settings live in version-controlled files.

20 simulated phones, on 2 network slices

A network slice is a way of splitting one physical network into separate virtual networks — so, for example, emergency services and normal customers can share hardware but stay isolated. We set up two real slices and split the devices between them:

Slice	Identifier	Devices	Address range
Slice A	SST 1 / SD 010203	10 devices	10.60.0.0/16
Slice B	SST 1 / SD 112233	10 devices	10.61.0.0/16

These are genuine 5G slices, not labels. Each device asks for its slice during connection, and the network's slice-selection function independently approves it.

The monitoring system

Prometheus collects measurements every 15 seconds; Grafana draws the dashboards. It runs as a completely separate system that only reads from the 5G core — the core doesn't even know it's being watched, and monitoring can never interfere with the network.

Automation

Scripts handle: downloading exact software versions, checking the machine is suitable, setting up SIM cards, launching devices, collecting evidence, and running the test suite. A CI pipeline checks on every code change that nobody has quietly swapped a software version.

3. Evidence — screenshots from the live system



Figure 1 — The main dashboard. Top row: 8 network functions being monitored, 20 devices connected, 0 active data sessions, 20 connection requests. The charts show device states and message rates over the last hour. Note "Active PDU sessions = 0" — that is expected and explained in section 5, not a fault.

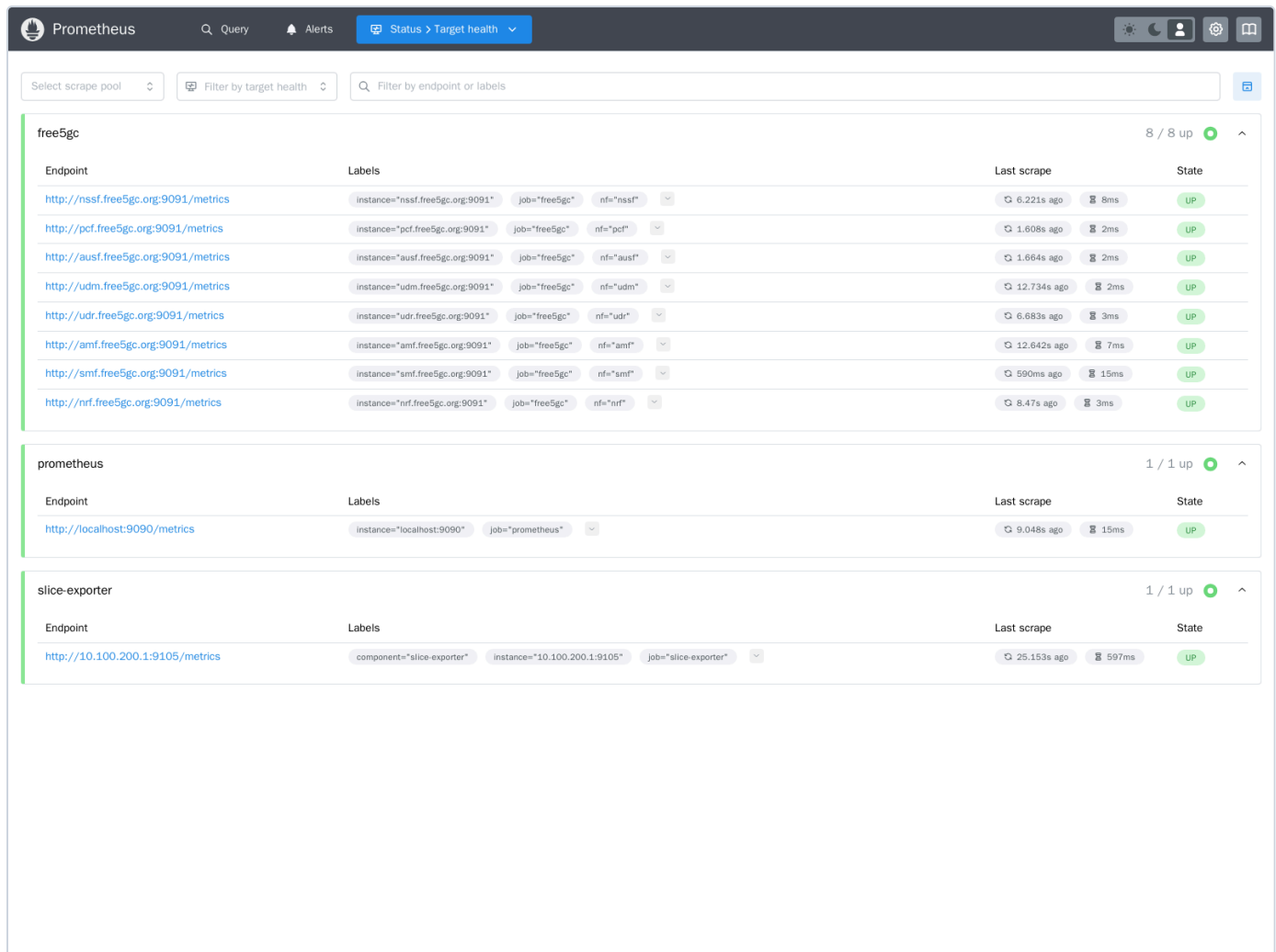


Figure 2 — Monitoring health. All 8 network functions reporting UP, plus Prometheus itself and our custom slice exporter. 10 healthy sources of data.

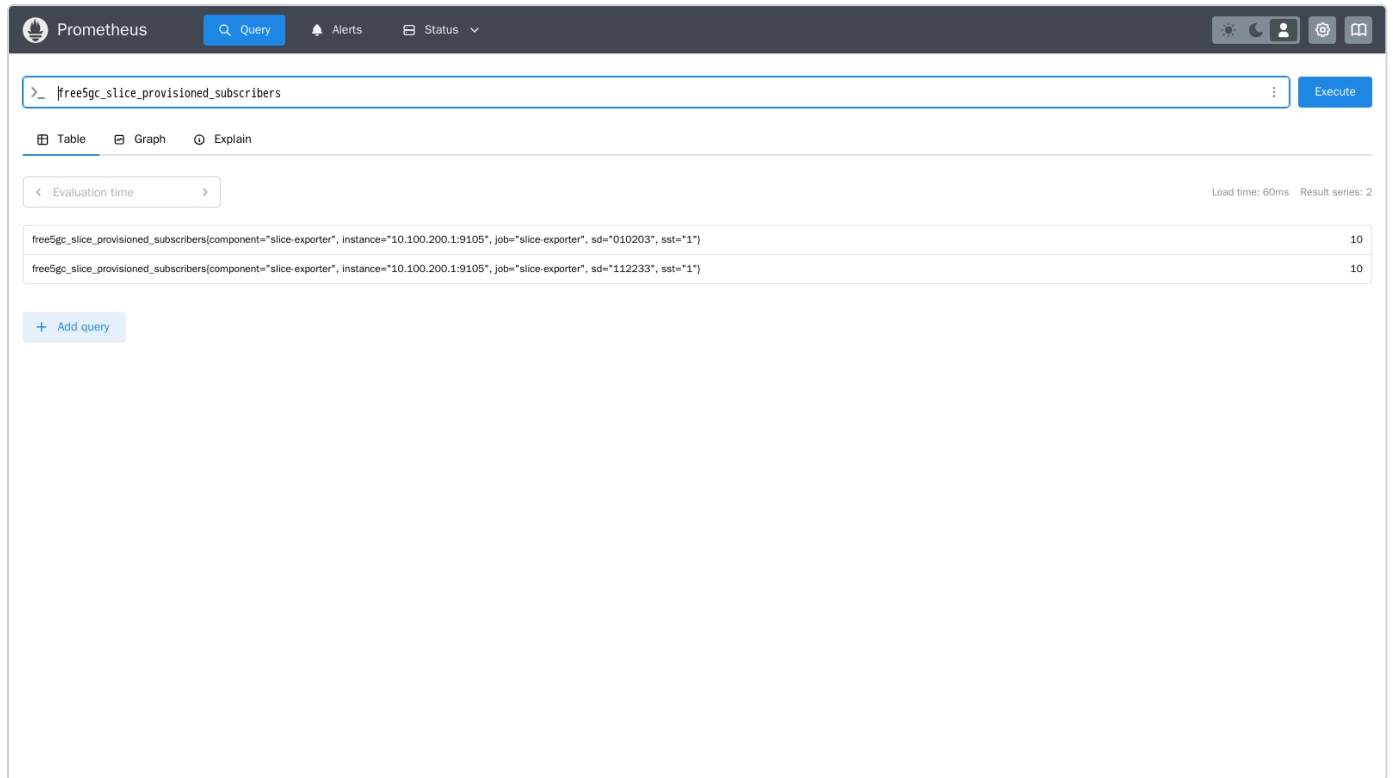


Figure 3 — The most important result. Measurements tagged by network slice: 10 subscribers on slice `010203` and 10 on slice `112233`. `free5GC` cannot produce this by itself (see section 4) — we had to build it.

4. What we discovered

4.1 The monitoring gap — and how we closed it

The biggest finding. We checked every single measurement free5GC produces — 629 of them — and found that none of them say which network slice they belong to.

That is a serious problem for this project, because the whole goal is to monitor networks *per slice*. Without slice labels you can see that the network is busy, but not *which slice* is busy. The project's design document had listed this as a risk; we confirmed it is real.

Our solution: the network's logs do mention the slice, even though the measurements don't. We wrote a small program that reads those logs, combines them with the subscriber database, and produces proper slice-tagged measurements — without modifying free5GC itself. Figure 3 is that solution working.

4.2 Some built-in counters can't be trusted

We found that free5GC's own device counters drift over time. Immediately after a clean start they are correct, but once connections fail and retry, they lose track and never recover:

Source	After a clean start	After lots of retries
Asking each device directly (the truth)	20	20
free5GC's "connected devices" counter	20 ✓	3 ✗
free5GC's "connected state" counter	20 ✓	63 ✗

Worth knowing before anyone builds alerts on those numbers. Our reports use the device's own answer as the source of truth.

4.3 Three faults blocked device connection — all misleading

1. A counter set too high. free5GC's default security counter was so far ahead of what a fresh device expects that every connection was rejected. *The error looked exactly like a wrong SIM key — the keys were correct the whole time.*
2. A missing location entry. The slice-selection function didn't know about our network's location area. The value also had to be written as text "000001", not the number 1 — otherwise it silently fails to match.
3. Devices asking for a slice they weren't allowed on. The default config asked for a second slice that no SIM was registered for, which made the network try to hand the device to a different server that doesn't exist.

4.4 Structured logging isn't possible on this version

The design asked for machine-readable (JSON) logs. We checked the source code: free5GC v4.2.3 simply doesn't offer that option. Recorded as a known limitation rather than modifying someone else's software.

4.5 The previous project's "slices" weren't real

The earlier version of this project appeared to do slice-based traffic handling. It didn't — it was a web app sorting requests with a lookup table and calling them "slices". The 5G core and the traffic simulator never talked to each other. The slicing in this project is real.

5. What is not finished, and why

One hardware limitation blocks the remaining work.

Moving actual user data through a 5G network needs a special Linux kernel component called gtp5g. It only works on a normal Linux machine. This laptop runs Linux inside Windows (WSL2), whose kernel cannot load it. This is not a configuration problem and cannot be worked around in software.

What works without it: everything that happens when a phone connects — authentication, security, slice selection, registration. That is the majority of a 5G core, and it is fully working here.

What doesn't: the data path. Devices connect successfully but cannot be given an IP address or send traffic. This is why "Active PDU sessions" reads 0 in Figure 1.

Remaining item	Why it's blocked
Load the gtp5g kernel component	Needs a standard Linux machine
Establish data sessions / assign IPs	Depends on the above
Prove end-to-end connectivity (ping)	Depends on the above
Record final hardware versions	Must be captured on the target machine
Full rebuild-from-empty proof	Should be done with the data path present
Tag the finished baseline	Can't declare it finished while data doesn't flow

Because of this, Phase 1 is deliberately not marked complete. The automated test suite enforces that honestly — it reports these items as "needs proper hardware" and refuses to say the project is finished. Running it on a suitable machine should turn them all green without code changes.

6. How to run it yourself

```
git clone https://github.com/RithvikReddy0-0/5g-observability.git
cd 5g-observability

bash scripts/verify_env.sh      # is this machine suitable?
bash scripts/bootstrap.sh      # download exact software versions

# start the core (details in docs/runbooks/clean-install.md)

bash scripts/provision_subscribers.sh --delete-all
COUNT=10 START=1 SD=010203 bash scripts/provision_subscribers.sh
COUNT=10 START=11 SD=112233 bash scripts/provision_subscribers.sh
COUNT=10 COUNT_B=10 TEMPO=600 bash scripts/start_ues.sh

bash tests/acceptance.sh      # run all the checks
bash scripts/collect_evidence.sh
```

One thing to remember: always re-create the SIM cards immediately before starting the devices. A security counter moves forward every time a device connects, and restarted devices start from zero — if they drift apart, connection fails in a way that looks like a wrong SIM key.

7. Where everything lives

Folder	Contents
docs/RESULTS.md	All measured results in detail
docs/evidence/	Six timestamped raw-data captures
docs/screenshots/	The images in this report
docs/runbooks/	Step-by-step setup instructions
docs/interfaces/	Reference for every network connection type
diagrams/	Diagrams of how a device connects
scripts/ , tests/	All automation and the test suite

Every number in this report was measured on a running system, not estimated. The raw captures behind each figure are in docs/evidence/ , so any claim here can be checked independently.